



CI/CD Integration & GitOps

Automating Kubernetes Delivery Pipelines — Module 13 of 13

Enterprise EKS Platform





Agenda

Concepts

- CI/CD fundamentals for Kubernetes
- GitOps principles and models
- Push vs pull-based deployments

Tools & Patterns

- ArgoCD deep dive
- FluxCD overview
- Pipeline tooling (GitHub Actions, Tekton)
- Deployment patterns and promotion strategies

Lab 13: GitOps with ArgoCD — declarative application delivery, sync, and rollback (~30 minutes)



CI/CD Fundamentals for Kubernetes

From code commit to running containers

Traditional CI/CD vs Cloud-Native CI/CD

Traditional CI/CD

- Deploy artifacts (JARs, binaries) to VMs
- Configuration management tools (Ansible, Chef)
- Mutable infrastructure — update in place
- Snowflake servers over time
- Rollback = redeploy previous version

Cloud-Native CI/CD

- Deploy container images to orchestrators
- Declarative manifests (YAML, Helm, Kustomize)
- Immutable infrastructure — replace, never patch
- Identical environments from images
- Rollback = point to previous image tag

Key shift: The artifact is a container image, and the desired state is a versioned manifest in Git.

Pipeline Stages for Kubernetes

End-to-End Pipeline Flow

Code Commit → Build → Test → Scan → Push → Deploy → Verify

- **Build** — compile code, build container image
- **Test** — unit tests, integration tests, contract tests
- **Scan** — vulnerability scanning, policy compliance
- **Push** — tag and push image to registry (ECR, GHCR)
- **Deploy** — update manifests, sync to cluster
- **Verify** — smoke tests, health checks, monitoring

Separation of concerns: CI builds and tests the artifact. CD deploys the artifact. GitOps manages the desired state.

Container Image Lifecycle



Build

- Multi-stage Dockerfiles
- Minimal base images (distroless, Alpine)
- Build cache optimization



Tag & Push

- Immutable tags (Git SHA, semver)
- Never use `:latest` in deployments
- Push to ECR / private registry



Scan & Sign

- Trivy / Snyk vulnerability scan
- Cosign for image signing
- Admission control for verified images



GitOps Principles

Git as the single source of truth for infrastructure

GitOps: Definition & Four Principles

What is GitOps?

An operational framework applying DevOps practices to infrastructure — Git as the single source of truth for declarative infrastructure and apps.

1. Declarative

The entire system is described declaratively (YAML manifests, Helm charts, Kustomize overlays)

2. Versioned & Immutable

The canonical desired state is versioned in Git, providing a complete audit trail

3. Pulled Automatically

Agents automatically pull the desired state and apply it to the cluster

4. Continuously Reconciled

Agents observe actual state, detect drift, and correct it automatically

Push vs Pull-Based Deployment Models

Push-Based (Traditional)

- CI/CD tool runs `kubectl apply`
- Requires cluster credentials in CI
- No drift detection after deploy
- Pipeline is the only deployment path
- Examples: Jenkins, GitHub Actions (direct)

Pull-Based (GitOps)

- Agent in cluster polls Git for changes
- No cluster credentials outside cluster
- Continuous drift detection and repair
- Git commit is the deployment trigger
- Examples: ArgoCD, FluxCD

Security advantage: Pull-based models eliminate the need to store cluster credentials in external CI systems — the agent only needs read access to Git.

Jenkins CI/CD Pipeline Integration



Jenkinsfile Pipeline Stages

- **Build** — compile code, build container image, push to ECR
- **Test** — unit tests, integration tests, code coverage
- **Scan** — image vulnerability scanning (Trivy, ECR scan)
- **Deploy** — `kubectl apply` or `helm upgrade` to EKS



Kubernetes Plugin

- Dynamic build agents launched as Pods in EKS
- Each build gets an isolated, ephemeral environment
- Pod templates define containers (maven, docker, kubectl)
- Scales to zero when idle — no persistent agent VMs

EKS integration: Jenkins authenticates to EKS via IRSA (IAM Roles for Service Accounts) or kubeconfig with a service account token — avoid storing long-lived cluster admin credentials.

From Jenkins Push to GitOps Pull

Hybrid Pattern: Jenkins CI + GitOps CD

Jenkins builds, tests, and scans — then updates an image tag in the GitOps repo. ArgoCD detects the change and deploys to EKS.

Best of both worlds: Jenkins handles CI (build, test, scan) while ArgoCD owns CD (deploy, reconcile, rollback) — no cluster credentials leave the cluster.

Git as Single Source of Truth

Repository Structure Patterns

Apps use a Kustomize **base/overlay** structure (per-environment overlays for dev, staging, production); shared **infrastructure** and per-**cluster** config get their own directories.

Every change goes through a pull request — code review, approval, and audit trail are built into the workflow.



ArgoCD

Declarative, GitOps continuous delivery for Kubernetes

ArgoCD Architecture

API Server

- Web UI dashboard
- gRPC/REST API
- RBAC enforcement
- SSO integration (OIDC, SAML)

Repo Server

- Clones Git repositories
- Renders manifests (Helm, Kustomize)
- Caches rendered output
- Stateless & scalable

Application Controller

- Watches live cluster state
- Compares desired vs actual
- Executes sync operations
- Reports health status

Sync Policies

Automated Sync

- Auto-deploy on Git commit
- No manual approval needed
- Ideal for dev/staging
- Optional for production

Self-Heal

- Revert manual `kubectl` changes
- Detect and fix drift
- Ensures Git remains truth
- Prevents configuration drift

Prune

- Delete resources removed from Git
- Prevents orphaned resources
- Use with caution in prod
- Combine with Foreground deletion

FluxCD Overview

A toolkit approach to GitOps on Kubernetes

FluxCD Architecture

Toolkit of Specialized Controllers

Source Controller

- Fetches from Git, Helm repos, S3
- Produces versioned artifacts
- Handles authentication

Kustomize Controller

- Applies Kustomize overlays
- Handles plain YAML too
- Dependency ordering

Helm Controller

- Manages HelmRelease CRDs
- Automated upgrades/rollbacks
- Values from ConfigMaps/Secrets

Additional controllers: Notification Controller (alerts & webhooks), Image Automation Controller (auto-update image tags in Git)

FluxCD vs ArgoCD Comparison

Feature	ArgoCD	FluxCD
Architecture	Monolithic application	Toolkit of controllers
UI	Rich web UI built-in	No built-in UI (Weave GitOps optional)
Multi-tenancy	Projects & RBAC	Namespace-scoped CRDs
Multi-cluster	ApplicationSets	Kustomization per cluster
Image automation	Argo Image Updater (separate)	Built-in image automation
CNCF Status	Graduated	Graduated

Both are excellent choices. ArgoCD excels with its UI and multi-cluster management. FluxCD excels with its lightweight, composable design and native image automation.

Pipeline Tools

CI platforms, image scanning, and policy enforcement

Jenkins X & Tekton

Tekton Pipelines

- Kubernetes-native CI/CD framework
- Pipelines run as Pods in the cluster
- CRDs: Task, Pipeline, PipelineRun
- Reusable task catalog (Tekton Hub)
- Kaniko builds images without a Docker daemon

Jenkins X

- Opinionated CI/CD for Kubernetes
- Built on Tekton underneath
- Automated environment promotion
- Preview environments per PR
- GitOps-native from the start

Image Scanning in CI

Trivy

- Open-source (Aqua Security)
- OS packages + language deps
- IaC scanning + SBOM generation
- Fail the pipeline on critical CVEs (`--exit-code 1`)

Snyk

- Commercial with free tier
- Deep language dependency analysis
- Fix suggestions and PR automation
- Continuous monitoring for new vulnerabilities

Policy as Code

OPA Gatekeeper

- Admission controller (Rego language)
- ConstraintTemplates + Constraints
- Example: require team/cost-center labels

Kyverno

- Kubernetes-native (YAML, no Rego)
- Validate, mutate, generate resources
- Example: disallow the `:latest` image tag

Recommended Patterns

Strategies for reliable, auditable delivery

Environment Promotion Strategies

Promotion Flow

Dev → **Staging** → **Pre-prod** → **Production**

Branch-Based Promotion

- Each environment maps to a branch
- PR from dev → staging → main
- Simple to understand
- Merge conflicts can be challenging

Directory-Based Promotion

- Single branch, overlay directories
- PR updates overlay path (e.g., image tag)
- No branch divergence
- Promoted via automated PRs

Recommended: Directory-based with Kustomize overlays on a single branch. Use automated PRs to promote image tags between environments.

Secrets Management

Sealed Secrets

- Encrypt with public key; decrypt only in-cluster
- Safe to commit to Git
- `kubeseal` CLI seals plain secrets

External Secrets Operator

- Sync from AWS SM / Vault
- Auto-rotation support
- `ExternalSecret` CRD syncs on an interval

SOPS

- Encrypts values, not keys -- KMS / PGP / age
- Keeps YAML keys readable for diffs

Rollback Strategies

Git Revert (Recommended)

- `git revert <commit>` creates a new commit
- Preserves complete history
- ArgoCD/Flux auto-syncs the revert

ArgoCD Manual Rollback

- Rollback via UI or CLI
- Temporary — next sync overwrites
- Use for emergencies only
- `argocd app history / rollback`

GitOps rollback = Git revert. Forward-only commit history provides a clear, auditable record.

CI/CD Security Best Practices

Pipeline Security

- Use OIDC for cloud auth (no long-lived keys)
- Pin action versions by SHA, not tag
- Restrict self-hosted runner permissions
- Require signed commits in GitOps repos
- Separate CI and CD credentials

Supply Chain Security

- Sign images with Cosign / Sigstore
- Generate and verify SBOMs
- Use admission policies to verify signatures
- Implement SLSA provenance
- Audit base image dependencies

Defense in depth: Scan in CI, enforce policies at admission, monitor at runtime. No single gate is sufficient.

Putting It All Together

Complete GitOps Pipeline

1. Developer pushes code to app repo
2. CI pipeline: build → test → scan → sign → push image
3. CI updates image tag in GitOps repo (automated PR)
4. PR reviewed and merged
5. ArgoCD/Flux detects change, syncs to cluster
6. Health checks verify deployment
7. Notification sent (Slack, PagerDuty)

Benefits achieved: Declarative, auditable, reproducible, self-healing, secure

Key repos: Application repo (source code) + GitOps repo (manifests) = clean separation of concerns



Module Summary

- GitOps: Git as single source of truth for declarative infrastructure
- ArgoCD and FluxCD reconcile cluster state with Git
- Directory-based promotion with Kustomize overlays avoids branch divergence
- Secrets: Sealed Secrets, ESO, or SOPS
- Rollback via git revert preserves audit history
- Supply chain security: scanning, signing, admission policies



Lab 13: GitOps with ArgoCD

Estimated Time: ~30 minutes

- Explore the ArgoCD UI and CLI
- Create an ArgoCD Application from a public Git repo
- Observe sync status and application health
- Make a change, trigger a sync, and perform a rollback
- *Optional:* Explore FluxCD reconciliation and drift detection

Key Takeaways

Remember These Points

1. Separate app repos from GitOps manifest repos
2. Enable drift detection and self-healing so clusters always match Git
3. Use OIDC for CI auth and pin action versions by SHA

? Questions & Discussion

Module 13: CI/CD Integration & GitOps

Final module — Course Wrap-Up